MOCB-WSP: Hybrid-Encoded Multi-Objective CNN-BiLSTM with Joint NAS and HPO for Wind Speed Prediction

Chen-qi Wang, *Student Member, IEEE,*

***Abstract*—The development of clean energy has become a global priority, with wind power serving as a mainstream renewable energy source. Accurate wind speed prediction (WSP) is critical for power generation scheduling, ensuring the economic reliability of power systems, and promoting clean energy penetration. However, existing deep learning forecasting models rely heavily on expert experience for manual architecture and hyperparameter design, leading to labor-intensive development and an inherent trade-off between prediction accuracy and model complexity. To address these issues, this paper proposes MOCB-WSP, a novel automated hybrid deep learning framework for wind speed forecasting based on the Non-dominated Sorting Genetic Algorithm II (NSGA-II). It integrates a convolutional neural network (CNN) and a bidirectional long short-term memory (BiLSTM) network to extract temporal features of wind speed series, and develops a fixed-length hybrid encoding scheme enabling the end-to-end joint optimization of neural architecture search (NAS) and hyperparameter optimization (HPO) in a single unified search space. An NSGA-II-based multi-objective evolutionary algorithm is adopted to optimize candidate models via a bi-objective minimization problem balancing prediction accuracy and model complexity. Extensive experiments are conducted on three real-world wind farm datasets, with five state-of-the-art manually designed models and one state-of-the-art neuroevolution-based model selected as baselines. The results demonstrate that the proposed MOCB-WSP framework delivers excellent comprehensive performance, yielding forecasting metrics superior to all baseline models while featuring substantially lower model complexity, thereby realizing the accuracy–complexity trade-off for practical wind speed prediction.**

***Index Terms*—Wind Speed Prediction (WSP), Multi-objective Optimization, Neural Architecture Search (NAS), Hyperparameter Optimization (HPO)**

I. Introduction

As documented in the Global Wind Energy Council (GWEC) Global Wind Report 2025, the global wind energy sector achieved 117 GW of new installed capacity in 2024, marking continued expansion into emerging geographic markets while solidifying its position as a core pillar of the worldwide energy transition. [1]. However, the inherent intermittency, randomness, and non-stationarity of wind speed hinder accurate wind speed forecasting, with the resulting prediction errors introducing severe challenges to grid operation security and wind farm economic scheduling. Large forecasting deviations directly cause unreasonable generation scheduling, increased wind curtailment, and reduce the reliability of power grid [2]. Therefore, accurate, reliable, and edge-deployable wind speed forecasting is the fundamental prerequisite to mitigate these challenges, critical for improving wind power penetration and power system economy.

In the initial stages of wind speed prediction (WSP), primary reliance was on statistical models, such as Autoregressive Integrated Moving Average (ARIMA) [3] and the Markov model [4]. With the advancement of machine learning, a host of methods were proposed, including logistic regression [5], Support Vector Machine (SVM) [6] and tree-based models like decision tree (DT) [7], have been applied in wind speed prediction. While these models have undeniably improved forecasting performance to a certain extent, they often exhibit limited feature extraction ability, insufficient fitting ability for complex nonlinear patterns, and lack satisfactory prediction accuracy. With the rapid development of deep learning techniques, deep neural networks have gained increasing attention in wind speed forecasting. Among them, Convolutional Neural Networks (CNN) [8] are suitable for extracting local fluctuation features from wind speed sequences, while Long Short-Term Memory (LSTM) [9] networks are suitable for capturing long-term temporal dependencies in wind speed series. Benefiting from their powerful ability in modeling nonlinearity, non-stationarity, and complex temporal dynamics, deep learning models have gradually become the mainstream approach for high-precision wind speed forecasting. However, most of the existing deep learning models utilized in wind speed forecasting are determined by manual design, which greatly rely on the experiences of designers. In addition, various machine learning or deep learning-based wind speed forecasting models have been proposed for different scenarios, [10], [11], [12], [13] e.g., onshore and offshore wind farms, ultra-short-term and short-term forecasting horizons [14], [15], [16], [17]. The neural architectures and hyper-parameters of these models are also manually designed.

Nowadays, the topic of performing neural architecture search (NAS) and hyperparameter optimization (HPO) automatically by evolutionary algorithms with a variety of encoding methods has become a research focus [18], [19]. To obtain a high-performance model for wind speed forecasting, a logical way is to design deep learning models through the combination of NAS and HPO. However, most literature focuses on NAS or HPO solely, since optimizing both simultaneously is computationally expensive and complex, and few existing

This work was supported by the National Natural Science Foundation of China (Grant No. XXX).

Chen-Qi Wang is with the School of Information and Intelligent Science, Donghua University, Shanghai 201620, China (e-mail: 240400123@mail.dhu.edu.cn).

(Corresponding author: XXX)

studies have focused on this joint optimization problem to date [20], [21], [22], [23]. On the other hand, current high-precision wind speed forecasting models face challenges for edge deployment in wind farms due to resource constraints, so objectives like model complexity should be well considered alongside prediction accuracy.Therefore, various automated deep learning models were presented by considering model complexity or robustness [24], [25], [26], [27], [28]. It is important to note that the majority of the aforementioned research on automated model design has been developed for computer vision tasks, and very few relevant studies have been conducted in the field of wind speed prediction.

To fill this gap, this paper proposes a novel *Multi-Objective NSGA-II Optimized CNN-BiLSTM for Wind Speed Prediction (MOCB-WSP)* framework specifically designed for wind speed forecasting task, which is critical for wind farm real-time control and grid frequency regulation. The framework unifies neural architecture search (NAS) and hyperparameter optimization (HPO) into an end-to-end multi-objective evolutionary pipeline, automatically generating lightweight yet high-performance hybrid CNN-BiLSTM models without manual intervention. The core contributions of this work are summarized as follows:

(i) To the best of our knowledge, this is the first attempt to develop a multi-objective automatic design method that jointly performs neural architecture search (NAS) and hyperparameter optimization (HPO) for wind speed prediction. The proposed MOCB-WSP framework is the first to achieve multi-objective joint NAS and HPO of hybrid CNN-BiLSTM architectures, effectively balancing prediction accuracy and model complexity for practical wind speed forecasting.
(ii) An efficient fixed-length hybrid encoding strategy is developed to simultaneously characterize the neural architecture, CNN/BiLSTM structural parameters, and key training hyperparameters such as learning rate, batch size, and optimizer settings. This unified encoding enables end-to-end joint optimization of the entire model configuration and substantially reduces the cost of manual parameter tuning.
(iii) Extensive experiments are conducted on three distinct real-world wind farm datasets. The results demonstrate that the proposed MOCB-WSP outperforms five representative hand-crafted models and one automatically designed forecasting method, while achieving a favorable trade-off between prediction performance and model lightweightness.

The rest of this paper is organized as follows. Section II reviews the related work. Section III presents the theoretical preliminaries. Section IV elaborates the detailed design of the proposed MOCB-WSP framework. Section V details the experimental settings, comprehensive comparison results, and in-depth analyses, as well as the limitations of this work and potential future directions. Finally, Section VI concludes the paper.

II. Related Works

A. Deep Learning-based Wind Speed Prediction

Khodayar and Wang proposed a spatial-temporal graph convolutional model (GCDLA) combining LSTM and rough set for short-term wind speed forecasting [17], which effectively captures spatial-temporal correlations and achieves better forecasting accuracy and robustness.Han et al. proposed a deep learning framework (VADTN) integrating intra-seasonal oscillations and NWP for 15-day wind power forecasting, and verified its effectiveness on three real wind farm cluster datasets [29].In [30],Liang et al. proposed WPFormer, a spatial-temporal graph Transformer for wind power forecasting, which achieves state-of-the-art performance on real-world datasets by addressing data anomalies and volatility.Zhang et al. proposed STD-UNet for refined grid wind speed and direction prediction in China [13], which integrates spatiotemporal coupling, nonparametric attention and hybrid loss, and realizes high accuracy, efficiency and transferability in wind forecasting.Zheng and Zhang proposed a stochastic recurrent encoder-decoder network (SREDNN) for multistep probabilistic wind power prediction [12], which introduces latent random variables to enhance distribution modeling and yields better accuracy and reliability.Other emerging deep learning methods, like transfer learning [31], and deep reinforcement learning [32], have also been applied in the field of wind speed prediction. Despite these applications, developing effective deep learning strategies for wind speed prediction still requires heavy reliance on human expertise and laborious experimental tuning.

B. Joint Optimization of Both NAS and HPO

To develop high-performance deep learning models, automated optimization of network structures and training configurations has become a research hotspot; Sun et al. [20] introduced genetic algorithms (GA) to automatically search for optimal network architectures and simultaneously optimize weight initialization schemes and activation functions for image classification tasks, with the constructed automated models achieving better prediction performance than manual design models on the MNIST [33] and CIFAR [34] datasets. On this basis, Baldominos et al. [21] combined GA with grammatical evolution to realize the synchronous design of CNN topologies and model parameters, Dong et al. [22] proposed AutoHAS, a reinforcement learning-driven framework, to implement the joint optimization of NAS and HPO and obtained higher precision than ResNet and EfficientNet on the CIFAR [34] and ImageNet [35] datasets, and Zimmer et al. [23] developed Auto-PyTorch Tabular, an AutoML framework that integrates multi-fidelity optimization, meta-learning and ensemble learning to collaboratively optimize network structures and hyperparameters. Different from the above studies that regard NAS and HPO as a single optimization objective, Guerrero-Viu et al. [36] first explored the joint optimization problem of NAS and HPO from the multi-objective perspective and verified the method effectiveness on the Oxford-Flowers [37] and Fashion MNIST datasets, while Mostafa et al. [38] adopted NSGA-II to conduct NAS and HPO for one-dimensional CNNs and applied

the optimized model to obstructive sleep apnea detection. It should be emphasized that the above automated deep learning methods are almost all oriented to computer vision scenarios, and there are still few related studies focusing on the joint optimization of NAS and HPO in the field of wind speed prediction.

C. Automated Deep Learning Considered Model Complexity

In the development of automated deep learning models, other significant objectives such as model complexity and robustness should be carefully considered in addition to model performance. In [24], Lu et al. proposed NSGANetV1, an NSGA-II-based multi-objective evolutionary NAS method that jointly optimizes classification accuracy and model complexity, achieving superior performance on multiple image classification benchmarks. Xue et al. proposed SMEMNAS [25], a multiobjective evolutionary NAS method with a multipopulation mechanism and pairwise comparison-based surrogate model, which balances classification accuracy and model complexity with high search efficiency on CIFAR [34] and ImageNet [35] datasets. In [26], Xue et al. proposed MOEA-BUS, a multiobjective evolutionary NAS method with bipopulation and uniform sampling mechanisms, which improves population diversity and search efficiency while balancing classification accuracy and model complexity on CIFAR [34] and ImageNet [35] datasets. Yang et al. proposed TRNAS [27], a training-free robust neural architecture search method with an R-Score zero-cost proxy and multi-objective selection strategy, which significantly reduces search costs and achieves state-of-the-art adversarial robustness. To tackle the joint optimization of neural architecture search and model robustness enhancement, Liu et al. [39] formalized this task as a multi-objective optimization problem, and further designed a corresponding bi-fidelity multi-objective NAS method. Wei et al. developed MoAR-CNN [28], an NSGA-II-based multi-objective robust NAS scheme for SAR image classification, which realizes joint optimization of network architectures and key hyperparameters, and achieves a superior balance between classification accuracy, adversarial robustness and model scale. Thus, it is necessary here to consider model complexity when developing automated deep learning models for wind speed forecasting tasks.

III. PRELIMINARIES

Before introducing the proposed MOCB-WSP framework for the ultra-short-term wind speed forecasting problem, this section briefly presents the preliminaries on one-dimensional Convolutional Neural Network (1D-CNN), Bidirectional Long Short-Term Memory (BiLSTM) network, and multi-objective optimization with the Non-dominated Sorting Genetic Algorithm II (NSGA-II).

A. 1D Convolutional Neural Network

Fig. 1 gives the structure of a one-dimensional CNN, which mainly consists of the input layer, convolutional layer, pooling layer, and fully connected layer. The core component

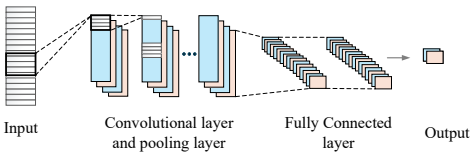


Fig. 1. Structure of the one-dimensional convolutional neural network (1D-CNN).

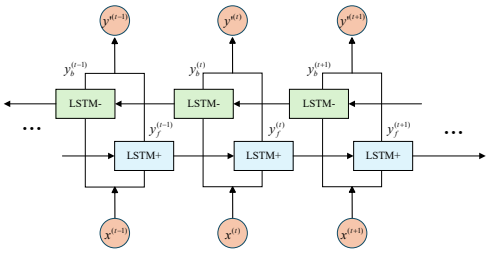


Fig. 2. Structure of the Bidirectional Long Short-Term Memory (BiLSTM) network.

is the convolutional layer, which performs sliding window convolution on the input wind speed sequence to capture local short-term fluctuation patterns.

The core 1D convolution operation is formulated as [40]:

$$C_j = \sigma\left(\sum A_i \otimes w_i + b_i\right) \tag{1}$$

where C_j is the output feature map of the convolutional kernel, A_i represents the input of the convolutional layer, the symbol $\otimes$ denotes the convolutional operation, σ represents the activation function, w_i and b_i are the weight matrix and the deviation, respectively.

The pooling layer is designed to downsample the feature map, reduce feature dimensionality and computational complexity. The fully connected layer aggregates the extracted local temporal features, which are then fed into the BiLSTM module for long-range temporal dependency modeling.

B. Bidirectional Long Short-Term Memory Network

Bidirectional Long Short-Term Memory (BiLSTM) is an improved recurrent network for time series modeling, which is developed from the basic Long Short-Term Memory (LSTM) network [41]. The structure of BiLSTM is illustrated in Fig. 2.

Different from unidirectional LSTM, BiLSTM contains a forward LSTM layer and a backward LSTM layer, which can model temporal dependencies from both past-to-future and future-to-past directions simultaneously. The output of BiLSTM is the concatenation of the hidden states from two directions, which provides a more comprehensive feature representation for non-stationary wind speed sequences.

As the basic unit of BiLSTM, the internal calculation formulas of LSTM are given as follows [42]:

$$f^{(t)} = \sigma\left(W_{fy}y^{(t-1)} + W_{fx}x^{(t)} + b_f\right) \tag{2}$$

$$i^{(t)} = \sigma\left(W_{iy}y^{(t-1)} + W_{ix}x^{(t)} + b_i\right) \tag{3}$$

$$u^{(t)} = \tanh\left(W_{uy}y^{(t-1)} + W_{ux}x^{(t)} + b_u\right) \tag{4}$$

$$c^{(t)} = i^{(t)} \cdot u^{(t)} + f^{(t)} \cdot c^{(t-1)} \tag{5}$$

$$o^{(t)} = \sigma\left(W_{oy}y^{(t-1)} + W_{ox}x^{(t)} + b_o\right) \tag{6}$$

$$y^{(t)} = o^{(t)} \cdot \tanh\left(c^{(t)}\right) \tag{7}$$

where $f^{(t)}$, $i^{(t)}$, $o^{(t)}$ represent the forget gate, input gate, output gate, respectively; $c^{(t)}$ is the cell state; $y^{(t)}$ is the hidden state output; W and b denote the weight matrices and bias vectors; $\sigma(\cdot)$ and $\tanh(\cdot)$ are the sigmoid and hyperbolic tangent activation functions.

On this basis, the final output of BiLSTM is calculated as:

$$y'^{(t)} = g\left(W_{y'f}y_f^{(t)} + W_{y'b}y_b^{(t)} + b_{y'}\right) \tag{8}$$

where $y_f^{(t)}$ and $y_b^{(t)}$ are the outputs of forward and backward LSTM layers, $W_{y'f}$, $W_{y'b}$ and $b_{y'}$ are the weights and bias, and $g(\cdot)$ is the output activation function.

By combining bidirectional information, BiLSTM can better capture the long-range temporal dependencies and periodic characteristics of wind speed series, which is beneficial to improve the forecasting performance of the model.

C. Multi-Objective Optimization and NSGA-II Algorithm

The standard mathematical formulation of a multi-objective optimization problem (MOP) is given as:

$$\begin{aligned} \min \quad & F(\tau) = (f_1(\tau), \ldots, f_z(\tau)) \\ \text{s.t.} \quad & \tau \in \Omega \end{aligned} \tag{9}$$

where $\tau = (\tau_1, \tau_2, \ldots, \tau_w)$ is a w-dimensional decision variable vector, $\Omega \subset \mathbb{R}^w$ denotes the w-dimensional feasible search space, $F : \Omega \to \mathbb{R}^z$ is a vector-valued objective function composed of z conflicting individual objectives $f_i(\tau)$ for $i = 1, 2, \ldots, z$. A decision vector $\rho = (\rho_1, \rho_2, \ldots, \rho_w) \in \mathbb{R}^w$ is said to dominate another vector $\varphi = (\varphi_1, \varphi_2, \ldots, \varphi_w) \in \mathbb{R}^w$, denoted as $\rho \prec \varphi$, if and only if $f_i(\rho) \leq f_i(\varphi)$ holds for all $i \in \{1, 2, \ldots, z\}$ and there exists at least one index $j \in \{1, 2, \ldots, z\}$ such that $f_j(\rho) < f_j(\varphi)$. A decision vector $\tau \in \Omega$ is defined as Pareto optimal (non-dominated) if no other vector $\tau^* \in \Omega$ exists such that $\tau^* \prec \tau$. The collection of all Pareto optimal solutions in Ω is called the Pareto-optimal set, and the corresponding set of objective values is referred to as the Pareto front.

As the most widely used evolutionary algorithm for solving MOPs, Non-Dominated Sorting Genetic Algorithm II (NSGA-II) was proposed by Deb et al. in 2002 [43], which has become the de facto benchmark for multi-objective optimization in engineering applications. Compared with traditional multi-objective algorithms, NSGA-II addresses the shortcomings of high computational complexity, poor diversity maintenance, and lack of elitist mechanism in early evolutionary algorithms.

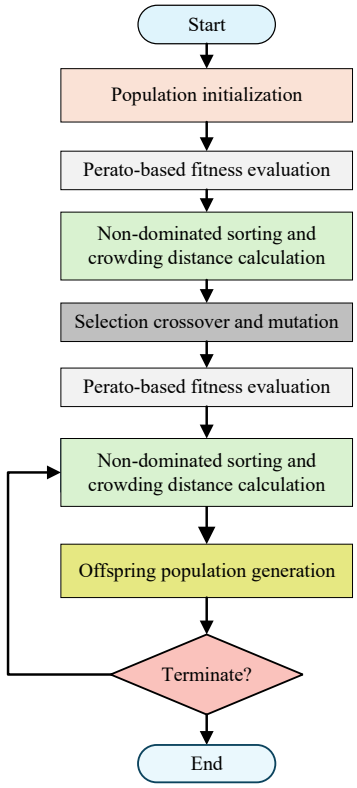


Fig. 3. Workflow of the NSGA-II multi-objective optimization algorithm.

It adopts three core mechanisms to achieve efficient and stable optimization: fast non-dominated sorting reduces the computational complexity from $O(mN^3)$ to $O(mN^2)$ by classifying individuals into hierarchical Pareto fronts; crowding distance calculation measures the distribution density of solutions in the objective space, ensuring the uniformity of the converged Pareto front; elitist preservation strategy combines parent and offspring populations for environmental selection, which accelerates convergence to the true Pareto optimal front and effectively avoids premature convergence to local optima. The workflow of NSGA-II is depicted in Fig. 3.

In this work, NSGA-II serves as the core optimization engine to jointly optimize the architectures and hyperparameters of the hybrid CNN-BiLSTM model, solving a bi-objective problem that balances prediction accuracy and model complexity to generate a set of Pareto-optimal trade-off solutions.

IV. The Proposed Method

In this section, the technical design and implementation details of the MOCB-WSP framework are systematically elaborated.

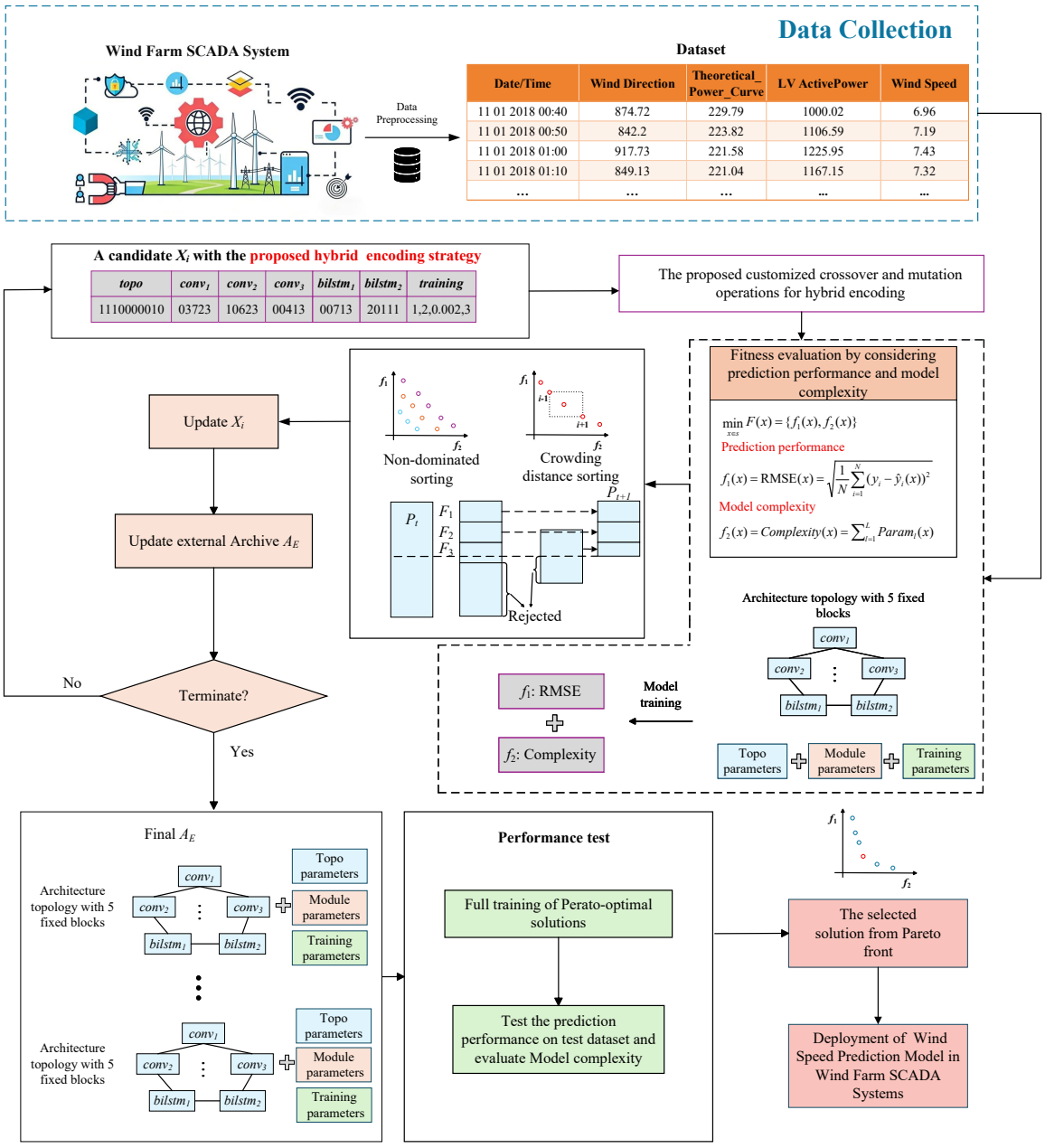


Fig. 4. The overall framework of the proposed MOCB-WSP.

A. Overview

Fig. 4 gives the overall framework of the proposed MOCB-WSP method. First, real supervisory control and data acquisition (SCADA)-based wind farm systems are used to gather the wind speed time series dataset. Then, the raw data is transformed into the standard time series dataset by data preprocessing and divided into the training set, validation set and test set. The proposed MOCB-WSP is performed on the training and validation datasets to find the optimized hybrid CNN-BiLSTM architectures and hyper-parameters by a customized NSGA-II. After the evolutionary process, the models in the final external archive are fully trained with related hyper-parameters for evaluation. The pseudo-code of MOCB-WSP is given in Algorithm 1. Firstly, generate an initialized parent population P_0 randomly by the proposed fixed-length hybrid encoding strategy, which means the neural architectures and hyper-parameters for a fixed-length blocks-based hybrid CNN-BiLSTM model, and initialize the external archive A_E by an empty set. Secondly, generate an offspring population Q_t by the proposed customized crossover and mutation methods based on P_t, and evaluate the bi-objective fitness of the combined population $R_t = P_t \cup Q_t$ by Pareto-based strategy with non-dominated sorting. Thirdly, update the external archive A_E with the non-dominated solutions from R_t by the proposed archive updating method, and select the next-generation population P_{t+1} via elitist environmental

selection. Repeat the second and third phase until the maximal generation time T. Finally, assess the model performance of wind speed forecasting based on the optimized hybrid CNN-BiLSTM with the non-dominated architectures and hyperparameters in A_E.

B. The Proposed Hybrid Encoding Strategy

The design of an effective encoding strategy is a fundamental challenge in neuroevolution-based neural architecture search (NAS), as it fundamentally governs the search space expressiveness, evolutionary operator validity, solution feasibility, and final model performance [44]. Existing encoding paradigms are mainly divided into variable-length and fixed-length types: the former allows dynamic adjustment of network blocks but suffers from inherent length drift during evolution, high probability of generating invalid solutions, and requires complex length-adaptive evolutionary operators [45], [46]; while the latter maintains constant individual dimensionality throughout the evolutionary process, simplifies crossover and mutation designs, eliminates invalid solutions at the encoding level, and exhibits superior search efficiency and stability, making it particularly suitable for time-sensitive and resource-constrained applications such as real-time wind speed forecasting. Furthermore, most existing NAS approaches only encode network topologies and module parameters while treating training hyperparameters as manually tuned constants, which inevitably leads to suboptimal solutions due to the synergistic interaction between architecture and training configuration [23]. Even the few evolutionary-based joint optimization studies adopt separate encoding schemes for discrete architectural parameters and continuous training parameters, compromising individual structural integrity and failing to fully exploit cross-parameter coupling relationships [36].

To address the above limitations, we propose a unified fixed-length 39-dimensional hybrid encoding scheme that seamlessly integrates discrete network topology, integer module parameters, and continuous training hyperparameters into a single mixed-variable vector, enabling end-to-end joint optimization of all critical components of the hybrid CNN-BiLSTM model. Specifically, we construct this encoding scheme for the hybrid CNN-BiLSTM backbone with 5 fixed independent blocks (3 CNN modules and 2 BiLSTM modules), which has been widely validated as an effective architecture for wind speed time series forecasting tasks. Each individual in the NSGA-II population is represented as a mixed-variable vector:

$$\boldsymbol{x} = [\boldsymbol{x}_{topo}, \boldsymbol{x}_{\text{CNN}}, \boldsymbol{x}_{\text{BiLSTM}}, \boldsymbol{x}_{\text{train}}] \tag{10}$$

where $\boldsymbol{x}_{topo}$ denotes the binary topology encoding segment, $\boldsymbol{x}_{\text{CNN}}$ represents the integer encoding for CNN module parameters, $\boldsymbol{x}_{\text{BiLSTM}}$ is the integer encoding for BiLSTM module parameters, and $\boldsymbol{x}_{\text{train}}$ contains the mixed-type encoding for training hyperparameters.

Fig. 5 illustrates the overall mapping process of the proposed encoding strategy, which converts a single 39-dimensional vector into a complete hybrid CNN-BiLSTM model with its corresponding training configuration. The following subsections detail the encoding rules for each segment.

1) Topology Encoding: The first 10 dimensions are reserved for binary topology encoding, which defines the directed feedforward connections among the 5 core modules labeled sequentially as Module 1 to Module 5. The topology encoding is formulated as:

$$\begin{aligned} \boldsymbol{x}_{\text{topo}} &= [\text{bin}_1, \text{bin}_2, \ldots, \text{bin}_{10}], \\ \text{bin}_i &= \text{Randint}(0, 1) \end{aligned} \tag{11}$$

The 10-bit binary codes are sequentially mapped to a 4×4 upper triangular adjacency matrix A to represent the network topology. For any element a_{ij} in matrix A, $a_{ij} = 1$ indicates a direct connection from Module i to Module $j + 1$, while $a_{ij} = 0$ means no such connection exists. A strict structural validity constraint is enforced to ensure that each module has at least one valid input connection.

2) CNN Module Parameter Encoding: The 11th to 25th dimensions (15 dimensions in total) encode the parameters of the 3 independent CNN modules, with 5 consecutive integer dimensions assigned to each module:

$$\begin{aligned} &\text{conv}_i = \{knum, ksize, kact, pt, ps\}, \\ &\begin{cases} knum = \text{Randint}(0, 7) \\ ksize = \text{Randint}(0, 3) \\ kact = \text{Randint}(0, 7) \\ pt = \text{Randint}(0, 2) \\ ps = \text{Randint}(0, 3) \end{cases} \end{aligned} \tag{12}$$

The corresponding mapping rules are defined as follows: $knum$: Actual number of output channels = $16 \times (knum + 1)$
$ksize$: Actual 1D convolutional kernel length = $2 \times ksize + 3$
$kact$: Activation function mapping:

$$kact = \begin{cases} 0: & \text{Softplus} \\ 1: & \text{Softsign} \\ 2: & \text{ELU} \\ 3: & \text{Softmax} \\ 4: & \text{Sigmoid} \\ 5: & \text{Tanh} \\ 6: & \text{ReLU} \\ 7: & \text{Linear} \end{cases} \tag{13}$$

pt: Pooling type mapping:

$$pt = \begin{cases} 0: & \text{MaxPooling} \\ 1: & \text{AvgPooling} \\ 2: & \text{No pooling layer} \end{cases} \tag{14}$$

ps: If $pt \neq 2$, actual pooling kernel length = $2 \times ps + 3$.

3) BiLSTM Module Parameter Encoding: The 26th to 35th dimensions (10 dimensions in total) encode the parameters

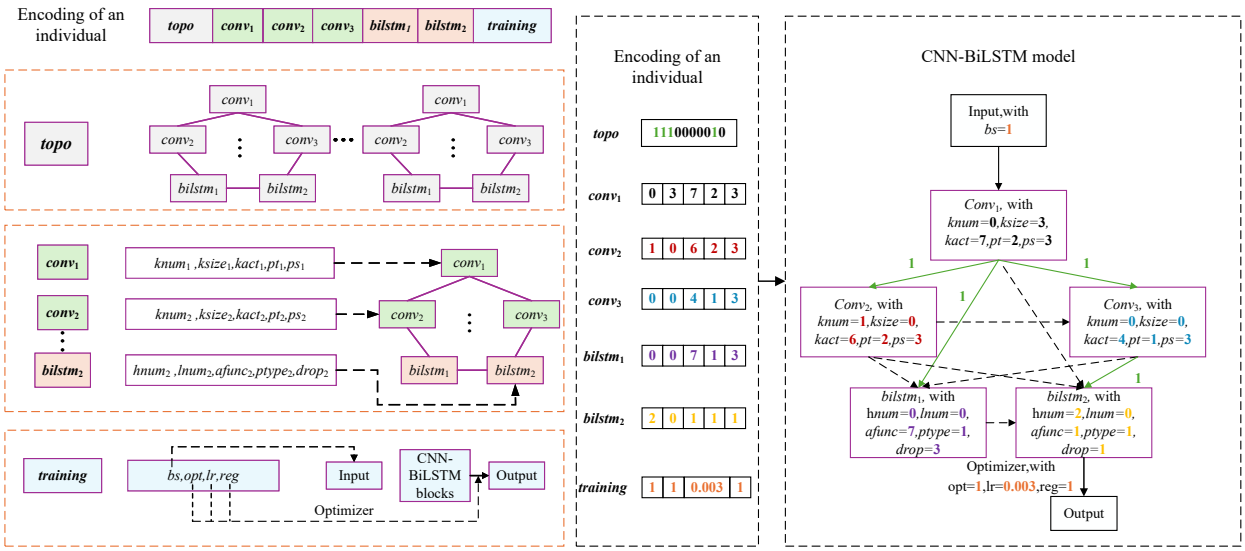


Fig. 5. Example of the proposed hybrid fixed-length encoding strategy for an initialized individual.

of the 2 independent BiLSTM modules, with 5 consecutive integer dimensions assigned to each module:

$$\text{bilstm}_j = \{hnum, lnum, afunc, ptype, drop\}, \begin{cases} hnum = \text{Randint}(0,7) \\ lnum = \text{Randint}(0,3) \\ afunc = \text{Randint}(0,7) \\ ptype = \text{Randint}(0,2) \\ drop = \text{Randint}(0,3) \end{cases} \tag{15}$$

where $hnum$ corresponds to the actual hidden size per direction $16 \times (hnum + 1)$, $lnum$ corresponds to the actual number of stacked layers $lnum + 1$, $afunc$ follows the same activation mapping as CNN modules, $ptype$ follows the same pooling mapping as CNN modules, and $drop$ corresponds to the actual dropout probability $drop \times 0.1$.

4) Training Hyperparameter Encoding: The final 4 dimensions (36th to 39th) encode the key training hyperparameters:

$$\boldsymbol{x}_{\text{train}} = \{bs, opt, lr, reg\}, \begin{cases} bs = \text{Randint}(0,3) \\ opt = \text{Randint}(0,3) \\ lr \in [10^{-4}, 10^{-2}] \\ reg = \text{Randint}(0,3) \end{cases} \tag{16}$$

where bs corresponds to the actual batch size $32 \times (bs + 1)$, and lr directly encodes the initial learning rate used in the training process. The optimizer mapping is:

$$opt = \begin{cases} 0: & \text{SGD} \\ 1: & \text{Adam} \\ 2: & \text{AdaDelta} \\ 3: & \text{RMSprop} \end{cases} \tag{17}$$

The regularization mapping is defined as:

$$reg = \begin{cases} 0: & \text{No regularization} \\ 1: & \text{L1 regularization} \\ 2: & \text{L2 regularization} \\ 3: & \text{L1\&L2 regularization} \end{cases} \tag{18}$$

C. Customized NSGA-II Evolutionary Process

NSGA-II is adopted as the core multi-objective optimization framework, with customized evolutionary operators designed for the hybrid encoding structure of the CNN-BiLSTM model. The initial population is constructed via constrained random initialization, ensuring each individual satisfies both structural connectivity and parameter bound constraints, as detailed in Algorithm 2.

In each generation, fast non-dominated sorting partitions the population into hierarchical Pareto fronts based on dominance relationships. A solution $\boldsymbol{x}_1$ dominates $\boldsymbol{x}_2$ if and only if $f_k(\boldsymbol{x}_1) \leq f_k(\boldsymbol{x}_2)$ for all $k \in \{1,2\}$ with at least one strict inequality, where f_1 denotes the validation RMSE and f_2 the total number of trainable parameters, both to be minimized.

To preserve population diversity and prevent premature convergence, the crowding distance for each individual is computed as:

$$d_i = \sum_{m=1}^{2} \frac{f_m^{(i+1)} - f_m^{(i-1)}}{f_m^{\max} - f_m^{\min}}, \tag{19}$$

where boundary individuals are assigned infinite crowding distance to guarantee retention of elite solutions.

Parent selection is performed via binary tournament selection based on Pareto rank and crowding distance. A segment-preserving single-point crossover operator is designed to maintain the structural integrity of each encoding segment, with

Algorithm 1 Overall Optimization Workflow of MOCB-WSP

Input: Population size N, maximum generation T, crossover probability p_c, mutation probability p_m, training set $\mathcal{D}_{train}$, validation set $\mathcal{D}_{val}$
Output: Final Pareto optimal set from external archive A_E
$P_0 \leftarrow$ InitializePopulation(N)
$A_E \leftarrow \emptyset$
Evaluate bi-objective fitness (f_1, f_2) for all individuals in P_0
Update A_E with non-dominated solutions from P_0
$t \leftarrow 0$
while $t < T$ **do**
    Perform fast non-dominated sorting and crowding distance calculation on P_t
    Select parent individuals via binary tournament selection
    $Q_t \leftarrow$ Crossover(parent population, p_c)
    $Q_t \leftarrow$ Mutation(Q_t, p_m)
    Evaluate bi-objective fitness of all individuals in Q_t
    $R_t \leftarrow P_t \cup Q_t$
    Perform fast non-dominated sorting on R_t
    Update A_E with non-dominated solutions from R_t
    Select top N individuals from R_t via elitist selection to form P_{t+1}
    $t \leftarrow t + 1$
end while
Extract final Pareto optimal set from converged A_E
return Final Pareto optimal set from A_E

Algorithm 2 Population Initialisation

Input: The population size N
Output: The initialised population P_0
$P_0 \leftarrow \emptyset$
for $it \leftarrow 1$ **to** N **do**
  $I \leftarrow$ Initialise an empty array for individual encoding
  setting $\leftarrow$ Randomly generate 4-integer array
  $I \leftarrow I \cup$ setting
  topo $\leftarrow$ Randomly generate 10-bit binary array
  $I \leftarrow I \cup$ topo
  for $j \leftarrow 1$ **to** 3 **do**
    $\text{cnn}_j \leftarrow$ Randomly generate 5-integer array
    $I \leftarrow I \cup \text{cnn}_j$
  end for
  for $j \leftarrow 1$ **to** 2 **do**
    $\text{lstm}_j \leftarrow$ Randomly generate 5-integer array
    $I \leftarrow I \cup \text{lstm}_j$
  end for
  $P_0 \leftarrow P_0 \cup I$
end for
return P_0

dedicated handling rules for the topology region. An example of the proposed crossover operation is shown in Fig. 6.

A variable-region mutation operator is further applied to enhance exploratory capability. In each mutation event, one of three encoding regions—topology, network module, or training hyperparameter—is randomly selected for regeneration, ensuring the mutated individual remains structurally valid.

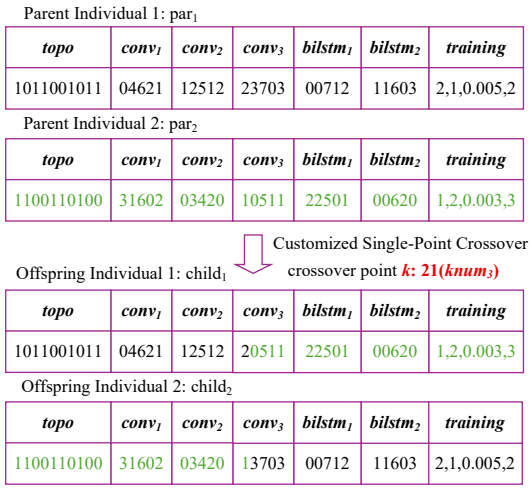


Fig. 6. An example of crossover operation.

Algorithm 3 Crossover Operation

Input: The parent population P, the crossover probability p_c, the population size N
Output: The offspring population Q
$Q \leftarrow \emptyset$
while $|Q| < N$ **do**
  Randomly select two individuals par_1, par_2 from P
  $r \leftarrow$ Randomly generate a real number from $(0, 1)$
  if $r < p_c$ **then**
    Randomly choose a crossover point k from 0 to 38
    if $k \in [0, 9]$ **then**
      $child_1 \leftarrow par_1$
      $child_2 \leftarrow par_2$
      Replace the topo in $child_1$ with a randomly generated topo
      Replace the topo in $child_2$ with a randomly generated topo
    else
      $p_1, p_2 \leftarrow$ Divide par_1 into two parts based on k
      $p_3, p_4 \leftarrow$ Divide par_2 into two parts based on k
      $child_1 \leftarrow p_1 \cup p_4$
      $child_2 \leftarrow p_2 \cup p_3$
    end if
  else
    $child_1 \leftarrow par_1$, $child_2 \leftarrow par_2$
  end if
  $Q \leftarrow Q \cup child_1, child_2$
end while
return Q

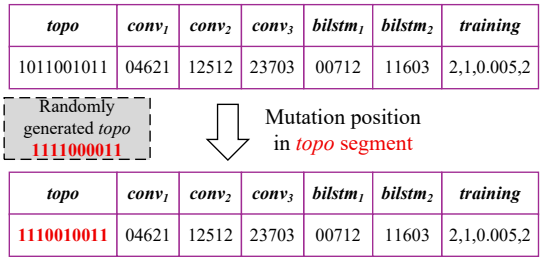


Fig. 7. Example of the proposed mutation operation on the topology region.

Algorithm 4 Variable-Region Mutation Operation

Require: Offspring population Q, mutation probability p_m
Ensure: Mutated population Q_M

$Q_M \leftarrow \emptyset$
for each individual $\boldsymbol{x} \in Q$ **do**
  $\boldsymbol{x}_{\text{tmp}} \leftarrow \boldsymbol{x}$
  $r \leftarrow \text{Random}(0, 1)$
  if $r < p_m$ **then**
    $R \leftarrow \text{RandChoice}(\{\text{Topo}, \text{Module}, \text{Setting}\})$
    if $R = \text{Topo}$ **then**
      new_topo $\leftarrow$ random 10-bit binary vector
      Replace topo segment in $\boldsymbol{x}_{\text{tmp}}$ with new_topo
    else
      Update $\boldsymbol{x}_{\text{tmp}}$ using a valid parameter in region R
    end if
  end if
  $Q_M \leftarrow Q_M \cup \{\boldsymbol{x}_{\text{tmp}}\}$
end for
return Q_M

Fig. 7 depicts an instance of the proposed mutation operation performed on the topology region.

An external archive A_E is maintained throughout evolution to store all non-dominated solutions. In each generation, the archive is updated by incorporating newly discovered non-dominated solutions and removing any dominated entries. When the archive size exceeds N, truncation based on crowding distance is applied to preserve solution diversity.

The next-generation population is determined via elitist environmental selection from the combined parent–offspring pool, prioritizing lower Pareto rank and, among equal-rank individuals, larger crowding distance. This mechanism ensures simultaneous convergence toward the true Pareto front and adequate coverage of the trade-off solution space.

D. Bi-Objective Fitness Evaluation

We formulate the automated design of hybrid CNN-BiLSTM models for wind speed forecasting as a bi-objective minimization problem:

$$\min_{\boldsymbol{x} \in \mathcal{F}} \boldsymbol{F}(\boldsymbol{x}) = [f_1(\boldsymbol{x}), f_2(\boldsymbol{x})]^\top \tag{20}$$

Algorithm 5 Bi-Objective Fitness Evaluation

Input: Encoded individual $\boldsymbol{x}$, training set $\mathcal{D}_{train}$, validation set $\mathcal{D}_{val}$
Output: Bi-objective fitness values (f_1, f_2)
Decode the topology, module parameters, and training hyperparameters from $\boldsymbol{x}$
Dynamically construct the hybrid CNN-BiLSTM model M with the decoded structure
Compute the total number of trainable parameters as $f_2 = \mathcal{C}(\boldsymbol{x})$
Train model M on $\mathcal{D}_{train}$ using the decoded hyperparameters, with early stopping to prevent overfitting
Generate wind speed predictions on the validation set $\mathcal{D}_{val}$
Calculate the validation RMSE as $f_1 = \text{RMSE}(\boldsymbol{x})$
return (f_1, f_2)

where the two objective functions are formally defined as follows:

$$f_1(\boldsymbol{x}) = \text{RMSE}(\boldsymbol{x}) = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i(\boldsymbol{x}))^2} \tag{21a}$$

$$f_2(\boldsymbol{x}) = \mathcal{C}(\boldsymbol{x}) = \sum_{i=1}^{L} \text{Param}_i(\boldsymbol{x}) \tag{21b}$$

Here, $f_1(\boldsymbol{x})$ denotes the root mean square error (RMSE) on the validation set, which quantifies the prediction accuracy of the model; $f_2(\boldsymbol{x})$ represents the total number of trainable parameters, which measures the model complexity; y_i and $\hat{y}_i(\boldsymbol{x})$ are the actual and predicted wind speed values, respectively; N is the number of validation samples; and $\text{Param}_i(\boldsymbol{x})$ is the number of trainable parameters in the i-th module of the model.

The detailed procedure of bi-objective fitness evaluation is outlined in Algorithm 5.

E. External Archive Update and Best-Compromise Solution Selection

The external archive A_E is designed to permanently preserve all non-dominated solutions generated during the entire evolutionary process, which represent the elite hybrid CNN-BiLSTM models balancing prediction accuracy and model complexity. The archive A_E is updated iteratively following three core rules: 1. First, all non-dominated solutions identified from the combined parent-offspring population R_t are added to A_E; 2. Subsequently, any existing solutions in A_E that are dominated by the newly added solutions are removed to strictly maintain the non-dominated property of the archive; 3. Finally, if the size of the archive exceeds the predefined population size N (i.e., $|A_E| > N$), A_E is truncated by eliminating individuals with the smallest crowding distances, thus preserving the diversity of the archived solutions in the objective space.

Upon the termination of the evolutionary process, the best-compromise solution is selected from the final converged archive A_E based on the normalized Euclidean distance to the ideal point in the bi-objective space. The ideal point is

defined as the hypothetical solution with the minimum values of both objectives. Formally, the best-compromise solution $\boldsymbol{x}^*$ is determined by:

$$\boldsymbol{x}^* = \arg\min_{\boldsymbol{x}\in A_E} \sqrt{\left(\frac{f_1(\boldsymbol{x}) - f_1^{\min}}{f_1^{\max} - f_1^{\min}}\right)^2 + \left(\frac{f_2(\boldsymbol{x}) - f_2^{\min}}{f_2^{\max} - f_2^{\min}}\right)^2} \tag{22}$$

where $f_1^{\min}$, $f_1^{\max}$, $f_2^{\min}$, and $f_2^{\max}$ denote the minimum and maximum values of the two objectives within the final archive A_E, respectively. The normalization of objectives ensures that both prediction accuracy and model complexity contribute equally to the distance metric. The best-compromise hybrid CNN-BiLSTM model corresponding to $\boldsymbol{x}^*$ is then fully trained on the entire dataset for final deployment.

V. Experimental Results

This section presents a systematic experimental evaluation of the proposed MOCB-WSP model using three real-world wind farm datasets. First, we introduce the datasets used and the corresponding data preprocessing pipeline. Then, we define the evaluation metrics and detail the experimental setup. Subsequently, we conduct comprehensive comparative analysis between the proposed model and representative state-of-the-art manually designed models. Finally, we present an in-depth analysis of the multi-objective optimization mechanism, generalization robustness, and computational overhead of the proposed automated modeling framework.

A. Datasets

To validate the prediction accuracy, generalization capability, and adaptability to different temporal resolutions of the proposed model, three real-world benchmark datasets from operational wind farms are employed. The key characteristics of the datasets are summarized in Table I, and detailed descriptions are as follows:

1 Sotavento Wind Farm 10-minute Dataset
This dataset was acquired from the SCADA system of the Sotavento Wind Farm in Galicia, Spain, with a 10-minute sampling interval [47], [48]. The input features include wind speed, wind direction, theoretical power curve, and low-voltage active power, with wind speed as the prediction target. The dataset is partitioned into training (70%), validation (15%), and test (15%) sets via time-series stratified partitioning to avoid data leakage.

2 Turkish Wind Farm 10-minute Dataset
This dataset is from an onshore wind farm in western Turkey, with a 10-minute sampling interval and the same four input features as the Sotavento dataset. It contains 1132 valid samples covering one week, capturing the diurnal variability of wind speed in this region. The same 70%/15%/15% partitioning strategy is used for consistency.

3 Turkish Wind Farm 1-hour Dataset
Derived from the same Turkish wind farm with a 1-hour sampling interval, this dataset contains 1000 valid samples over six weeks. The coarser temporal resolution increases the non-stationarity of the wind speed series, making prediction more challenging. The same partitioning strategy is maintained for cross-resolution comparison.

The key statistical characteristics (max, median, min, mean, standard deviation) of wind speed in each dataset are summarized in Tables II, III, and IV, and the wind speed time series are visualized in Figs. 8, 9, and 10.

TABLE II
Statistical Characteristics of the Sotavento 10-minute Dataset (units: m/s)

Dataset	Max	Median	Min	Mean	St.d
Entire dataset	16.29	6.84	0.29	7.28	3.87
Training dataset	16.29	6.41	0.29	6.63	3.77
Validation dataset	16.05	11.59	2.95	10.65	3.11
Test dataset	15.36	6.62	1.67	6.93	3.26

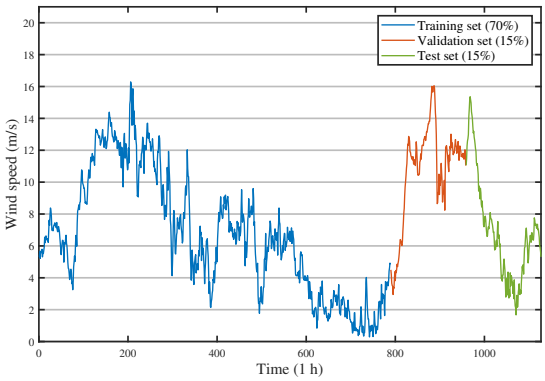


Fig. 8. Actual wind speed data of the Sotavento 10-minute dataset (Train/Val/Test split).

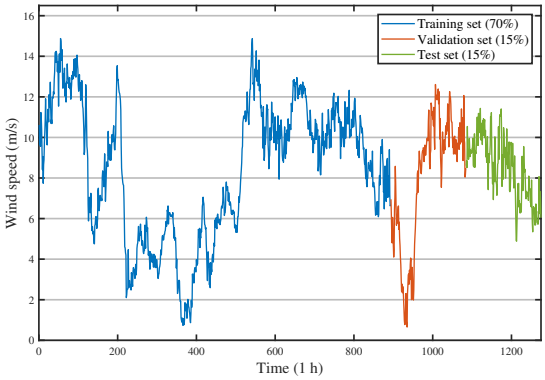


Fig. 9. Actual wind speed data of the Turkish 10-minute dataset (Train/Val/Test split).

B. Data Preprocessing

A unified and rigorous data preprocessing pipeline is implemented for all datasets to ensure data quality and experimental fairness:

1. Quality Control: Invalid samples (e.g., negative wind speed) are removed, missing values are supplemented via linear

TABLE I
STATISTICAL CHARACTERISTICS OF THE ADOPTED DATASETS

Dataset Name	Sampling Interval	Total Samples	Number of Features	Time Span
Sotavento Wind Farm	10 minutes	1128	4	1 week
Turkish Wind Farm (10-min)	10 minutes	1132	4	1 week
Turkish Wind Farm (1-hour)	1 hour	1000	4	6 weeks

TABLE III
STATISTICAL CHARACTERISTICS OF THE TURKISH 10-MINUTE DATASET (UNITS: M/S)

Dataset	Max	Median	Min	Mean	St.d
Entire dataset	14.87	9.04	0.65	8.36	3.14
Training dataset	14.87	9.13	0.73	8.43	3.37
Validation dataset	12.61	9.18	0.65	7.94	3.22
Test dataset	11.43	8.69	4.88	8.49	1.57

TABLE IV
STATISTICAL CHARACTERISTICS OF THE TURKISH 1-HOUR DATASET (UNITS: M/S)

Dataset	Max	Median	Min	Mean	St.d
Entire dataset	24.09	8.62	0.44	9.09	5.04
Training dataset	24.09	8.83	0.52	9.30	4.91
Validation dataset	24.04	8.58	2.37	9.70	5.55
Test dataset	18.29	6.79	0.44	7.47	4.84

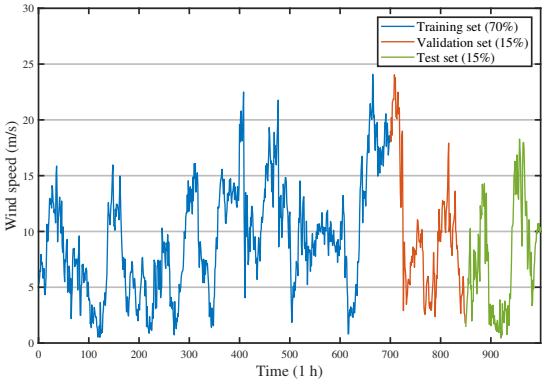


Fig. 10. Actual wind speed data of the Turkish 1-hour dataset (Train/Val/Test split).

interpolation, and outliers are eliminated using the 3σ criterion.

2. Normalization: All input features are normalized to the range [0,1] using Min-Max scaling to avoid feature scaling bias:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{23}$$

where $x_{\min}$ and $x_{\max}$ are the minimum and maximum values of the feature, respectively.

3. Time-Series Partitioning: The dataset is partitioned into training (70%), validation (15%), and test (15%) sets in chronological order to preserve the temporal distribution characteristics of wind speed data and avoid data leakage.

C. Metrics

To quantitatively evaluate the performance of wind speed prediction models from both accuracy and model complexity perspectives, five metrics are adopted in this work. The first four metrics, i.e., Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Mean Absolute Percentage Error (MAPE), and Pearson Correlation Coefficient (R), are specifically used to assess the wind speed prediction accuracy, while the last metric, Number of Trainable Parameters (Params), is employed to measure the model's computational complexity and lightweightness. For MAE, RMSE, and MAPE, lower values indicate better prediction accuracy. A value of R close to 1 signifies a stronger linear correlation between the predicted and actual wind speed sequences, reflecting the model's ability to capture the temporal variation trend of wind speed. The mathematical definitions of these four prediction accuracy metrics are given as follows:

$$\begin{aligned}
\text{MAE} &= \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \\
\text{RMSE} &= \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \\
\text{MAPE} &= \frac{100}{N}\sum_{i=1}^{N}\frac{|y_i - \hat{y}_i|}{y_i} \\
R &= \frac{\sum_{i=1}^{N}(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^{N}(y_i - \bar{y})^2}\sqrt{\sum_{i=1}^{N}(\hat{y}_i - \bar{\hat{y}})^2}}
\end{aligned} \tag{24}$$

where N denotes the total number of wind speed samples, y_i and $\hat{y}_i$ represent the i-th actual and predicted wind speed values respectively, and $\bar{y}$ and $\bar{\hat{y}}$ are the mean values of the actual and predicted wind speed sequences.

The number of trainable parameters quantifies the model's computational complexity and memory footprint, which is a fundamental concern in deep learning [49]. Fewer parameters correspond to a more lightweight model with lower memory consumption and faster inference speed, which is critical for real-time wind speed prediction deployed on edge devices and wind farm SCADA systems. Its mathematical expression is:

$$\text{Params} = \sum_{i=1}^{L}\text{Param}_i \tag{25}$$

where L denotes the total number of layers in the model, and Param_i represents the number of trainable parameters in the i-th layer.

D. Compared With Other Wind Speed Prediction Methods

This section presents comprehensive experimental results and comparisons with state-of-the-art wind speed prediction models to validate the effectiveness, efficiency, and generalization capability of the proposed MOCB-WSP framework. All experiments are implemented on three real-world wind speed datasets, namely the Sotavento 10-min dataset, the Turkish 10-min SCADA dataset, and the Turkish 1-h SCADA dataset. Five representative manually designed models with carefully hand-tuned hyperparameters from their original publications are selected as baseline methods, including CNN [50], LSTM [51], BiLSTM [52], CNN-BiLSTM [53], and LSTM-Transformer [54]; additionally, a typical neuroevolution-based approach, i.e., GA-GLSTM [55], is also adopted for comprehensive comparison. The prediction performance is evaluated by five widely accepted metrics: mean absolute error (MAE), root mean square error (RMSE), mean absolute percentage error (MAPE), correlation coefficient (R), and the number of trainable parameters (Params) for quantifying model complexity.

For the multi-objective architecture optimization, the NSGA-II algorithm is configured with a population size of 60 and a maximum of 30 generations, using crossover and mutation probabilities of 0.8 and 0.5, respectively. All optimizations and validations are performed for one-step-ahead wind speed time series prediction with a prediction step size of 1. During the search stage, each candidate architecture is trained for 20 epochs with an early stopping patience of 5 epochs, while the final optimized model undergoes comprehensive retraining for 200 epochs with an extended patience of 20 epochs. The best trade-off hybrid encoding vectors of MOCB-WSP for different datasets are presented in Table V.

All experiments are conducted on a personal laptop equipped with an Intel Core i7-13700H CPU, an NVIDIA GeForce RTX 4070 Laptop GPU (8 GB dedicated graphics memory), and 16 GB RAM. All models are implemented using Python 3.9.25 and PyTorch 2.5.1. To eliminate the impact of random parameter initialization, all models are independently trained and evaluated five times under identical experimental settings. The optimal result from these five independent runs is selected as the final reported performance for each model.

Table VI presents the quantitative comparison results on the Sotavento 10-min dataset, with optimal metric values highlighted in bold. The best trade-off solution of the proposed MOCB-WSP achieves an MAE of 0.4215 m/s, an RMSE of 0.5052 m/s, and a correlation coefficient R of 0.9871 using only 16.9×10^3 trainable parameters. The best performance solution attains the optimal prediction metrics, with an MAE of 0.3924 m/s, an RMSE of 0.5035 m/s, and an R of 0.9875. Compared with the manually designed CNN-BiLSTM baseline, MOCB-WSP significantly reduces model complexity while improving prediction accuracy, demonstrating the effectiveness of joint neural architecture and hyperparameter optimization for lightweight model design.

Table VII reports the experimental results on the Turkish 10-min SCADA dataset. Consistent with the Sotavento results, the proposed MOCB-WSP outperforms all hand-crafted and neuroevolution-based baselines. Specifically, the best trade-off solution achieves an MAE of 0.6605 m/s, an RMSE of 0.8428 m/s, a MAPE of 8.05%, and an R of 0.8554 with only 14.5×10^3 parameters. The best performance solution further improves prediction accuracy, yielding the lowest MAE of 0.6598 m/s and RMSE of 0.8243 m/s. These results validate the strong generalization capability of the proposed framework across diverse wind farm conditions.

Table VIII illustrates the performance on the Turkish 1-hour dataset, which exhibits higher non-stationarity and prediction difficulty due to the longer sampling interval. Even in this challenging scenario, MOCB-WSP achieves state-of-the-art performance. The best trade-off solution obtains the lowest MAE of 1.4470 m/s with only 13.6×10^3 parameters, while the best performance solution achieves the optimal RMSE of 1.9322 m/s and the highest R of 0.9163. These results further verify the superiority of MOCB-WSP for long-term wind speed forecasting.

Fig. 11 compares the predicted and measured wind speed profiles of the best trade-off solution across all datasets. The predicted profiles closely match the measured values under both short-term and long-term forecasting settings, with no noticeable systematic bias. These qualitative results further confirm that MOCB-WSP effectively models the nonlinear dynamics of wind speed while maintaining an ultra-compact structure, which is advantageous for real-time edge deployment in wind energy systems.

Fig. 12 illustrates the evolutionary trajectories of the Pareto-optimal fronts (POFs) at generations $t = 1$, $t = 5$, $t = 10$, $t = 20$, and $t = 30$ on the Sotavento 10-min and Turkish 1-hour datasets, which visualizes the multi-objective search process toward optimal configurations of the hybrid CNN-BiLSTM architecture, module parameters, and training hyperparameters. As can be observed, the POFs of both datasets gradually converge toward the lower-left ideal region through the iterative evolution of the customized NSGA-II. Specifically, the initial POF at $t = 1$ exhibits relatively large prediction error and model complexity. As evolution proceeds, the non-dominated solutions obtained at $t = 30$ exhibit clear dominance over those from earlier generations, indicating the strong global exploration and local exploitation capabilities of the proposed framework. The consistent evolutionary tendencies verify that the hybrid encoding strategy and tailored evolutionary operators enable the customized NSGA-II to generate well-distributed and well-converged POFs. Accordingly, MOCB-WSP can automatically identify lightweight yet high-precision wind speed prediction models while achieving a favorable balance between forecasting accuracy and model complexity, which effectively reduces the heavy dependence on manual expert experience and empirical trial-and-error in practical wind farm applications.

In summary, extensive experimental results demonstrate that the proposed MOCB-WSP outperforms state-of-the-art hand-crafted models and neuroevolution-based approaches in both prediction accuracy and model lightweight. By jointly

TABLE V
BEST TRADE-OFF HYBRID ENCODING VECTORS OBTAINED BY THE PROPOSED MOCB-WSP

Dataset	*Topo*	CNN	BiLSTM	Setting
Sotavento 10-min Dataset	[1,1,1,1,0,0,0,0,0,0]	[0,0,1,0,1] [1,3,4,1,1] [3,0,3,1,1]	[0,0,7,1,0] [0,0,0,1,2]	[0, 0, 0.0072, 3]
Turkish 10-min Dataset	[1,0,0,0,1,1,0,0,0,1]	[0,1,7,1,3] [0,0,6,0,1] [2,0,6,1,3]	[0,0,2,2,2] [0,0,7,1,2]	[0, 3, 0.0002, 3]
Turkish 1-hour Dataset	[1,1,0,1,0,1,0,0,0,0]	[0,2,2,0,3] [0,2,7,0,1] [2,0,5,0,3]	[0,0,6,0,2] [0,0,2,0,1]	[3, 0, 0.0053, 1]

TABLE VI
PERFORMANCE COMPARISON ON THE SOTAVENTO 10-MIN DATASET

	Model	MAE	RMSE	MAPE (%)	R	Params ($\times 10^3$)
Manually designed models	CNN [50]	0.5169 (7)	0.6722 (7)	**7.25 (1)**	0.9835 (6)	273.0 (8)
	LSTM [51]	0.4269 (5)	0.5386 (6)	8.28 (4)	0.9863 (5)	144.6 (6)
	BiLSTM [52]	0.4262 (4)	0.5329 (5)	8.36 (6)	0.9867 (3)	135.2 (5)
	CNN-BiLSTM [53]	0.4344 (6)	0.5284 (4)	10.56 (8)	0.9465 (8)	80.4 (3)
	LSTM-Transformer [54]	0.5461 (8)	0.7474 (8)	9.88 (7)	0.9743 (7)	95.5 (4)
Neuroevolution-based model	GA-GLSTM [55]	0.4217 (3)	0.5222 (3)	8.31 (5)	0.9865 (4)	210.2 (7)
Ours	MOCB-WSP (Best Trade-off)	0.4215 (2)	0.5052 (2)	8.12 (3)	0.9871 (2)	**16.9 (1)**
	MOCB-WSP (Best Performance)	**0.3924 (1)**	**0.5035 (1)**	7.99 (2)	**0.9875 (1)**	24.4 (2)

TABLE VII
PERFORMANCE COMPARISON ON THE TURKISH 10-MIN DATASET

	Model	MAE	RMSE	MAPE (%)	R	Params ($\times 10^3$)
Manually designed models	CNN [50]	0.6851 (7)	0.8649 (6)	10.92 (8)	**0.9402 (1)**	273.0 (7)
	LSTM [51]	0.6836 (6)	0.8814 (7)	8.33 (6)	0.8413 (6)	144.6 (6)
	BiLSTM [52]	0.6726 (3)	0.8559 (4)	8.18 (4)	0.8453 (4)	135.2 (5)
	CNN-BiLSTM [53]	0.6766 (5)	0.8488 (3)	8.21 (5)	0.8424 (5)	80.4 (3)
	LSTM-Transformer [54]	0.7679 (8)	0.9925 (8)	9.57 (7)	0.8016 (8)	95.5 (4)
Neuroevolution-based model	GA-GLSTM [55]	0.6754 (4)	0.8587 (5)	8.16 (3)	0.8394 (7)	955.4 (8)
Ours	MOCB-WSP (Best Trade-off)	0.6605 (2)	0.8428 (2)	**8.05 (1)**	0.8554 (3)	**14.5 (1)**
	MOCB-WSP (Best Performance)	**0.6598 (1)**	**0.8243 (1)**	8.15 (2)	0.8616 (2)	34.6 (2)

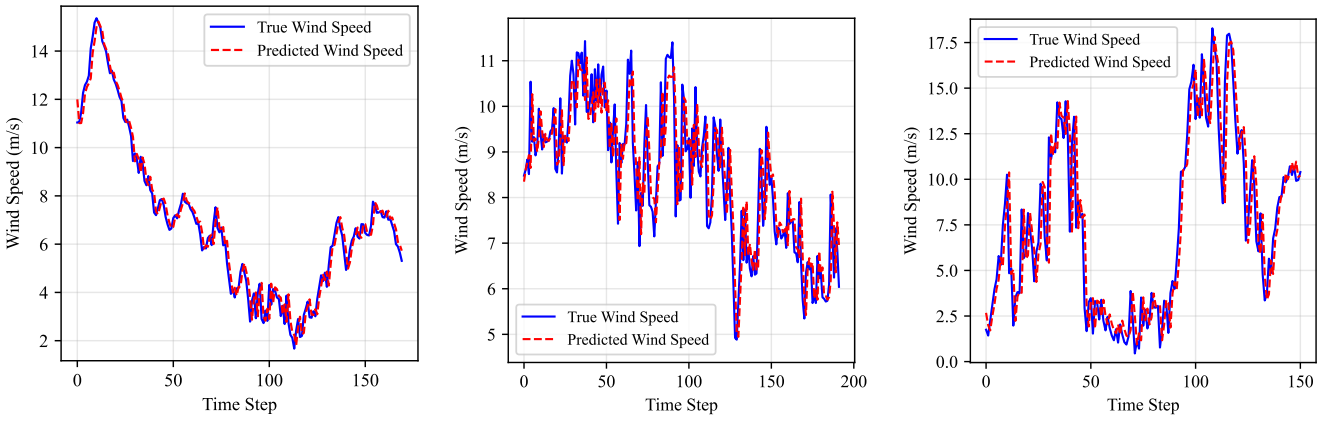
optimizing network topology, module parameters and training hyperparameters via multi-objective evolutionary algorithm, MOCB-WSP automatically constructs high-performance prediction models without manual expert intervention. Such advantages make the proposed framework a promising solution for reliable and real-time wind speed prediction in wind farms.

E. Discussion

From Tables VI, VII, and VIII, the proposed automated deep learning-driven wind speed forecasting framework, MOCB-WSP, demonstrates superior overall performance compared with state-of-the-art competitors in terms of prediction error metrics (MAE, RMSE, and MAPE), Pearson correlation coefficient R, and model lightweightness. This advantage stems from the NSGA-II-based multi-objective optimization mechanism, which autonomously explores high-performance hybrid CNN-BiLSTM architectures and effectively mitigates the inherent limitations of manually designed models, such as over-parameterization and heavy reliance on expert knowledge. By leveraging the powerful global search capability of the evolutionary algorithm, together with the proposed hybrid encoding strategy and customized evolutionary operators, MOCB-WSP achieves more competitive performance

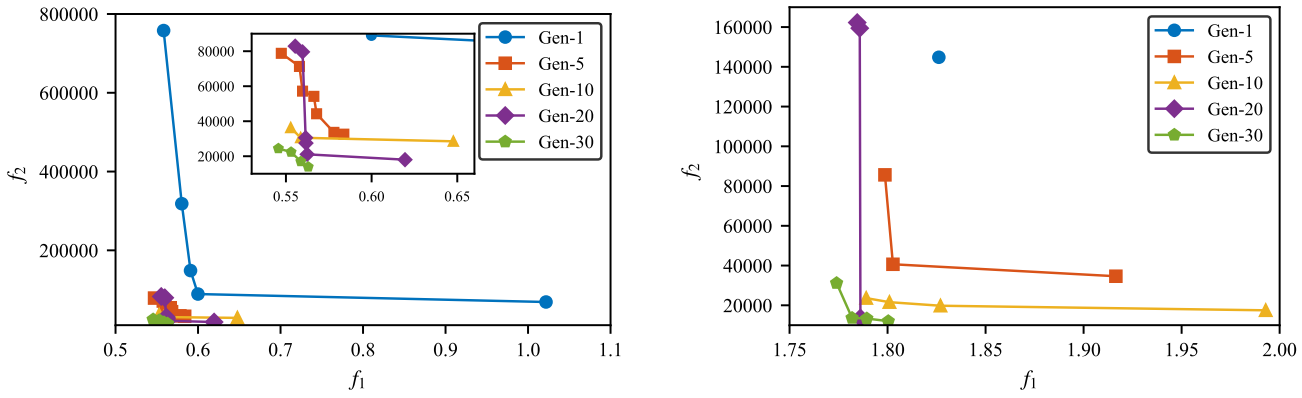
TABLE VIII
PERFORMANCE COMPARISON ON THE TURKISH 1-HOUR DATASET

	Model	MAE	RMSE	MAPE (%)	R	Params ($\times 10^3$)
Manually designed models	CNN [50]	1.5851 (7)	2.0925 (7)	34.33 (3)	0.9068 (4)	273.0 (8)
	LSTM [51]	1.4694 (3)	2.0412 (4)	33.76 (2)	0.9129 (3)	144.6 (6)
	BiLSTM [52]	1.5163 (6)	2.0512 (5)	36.51 (6)	0.9051 (5)	135.2 (5)
	CNN-BiLSTM [53]	1.4945 (5)	2.0551 (6)	39.69 (7)	0.9038 (6)	80.4 (3)
	LSTM-Transformer [54]	1.6909 (8)	2.2280 (8)	39.79 (8)	0.8870 (7)	95.5 (4)
Neuroevolution-based model	GA-GLSTM [55]	1.4865 (4)	2.0033 (3)	**30.04 (1)**	0.8703 (8)	243.7 (7)
Ours	MOCB-WSP (Best Trade-off)	**1.4470 (1)**	1.9660 (2)	35.09 (5)	0.9137 (2)	**13.6 (1)**
	MOCB-WSP (Best Performance)	1.4492 (2)	**1.9322 (1)**	35.04 (4)	**0.9163 (1)**	31.1 (2)



(a) Sotavento 10-min (b) Turkish 10-min (c) Turkish 1-hour

Fig. 11. Predicted and measured wind speed profiles of the best trade-off solution on the three test datasets: (a) Sotavento 10-min, (b) Turkish 10-min, and (c) Turkish 1-hour.



(a) Sotavento 10-min dataset (b) Turkish 1-hour dataset

Fig. 12. Evolutionary trajectories of Pareto-optimal fronts across representative generations on two wind speed datasets: (a) Sotavento 10-min dataset, (b) Turkish 1-hour dataset, where $t = i$ denotes the i-th generation.

than conventional neuroevolution-based forecasting methods. Furthermore, the framework exhibits strong generalization across diverse wind farm datasets with varying sampling intervals and autonomously identifies models that optimally balance accuracy and complexity, thereby facilitating practical deployment in real-world wind speed prediction applications.

The proposed MOCB-WSP has the following limitations. First, it searches a large hybrid space that encompasses both neural architectures and hyperparameters, rendering the optimization process computationally intensive and time-consuming. Second, as a purely data-driven deep learning method, MOCB-WSP offers limited interpretability and trans-

parency, which complicates detailed analysis of its decision-making mechanisms for wind speed prediction. Third, the neural architecture search is performed entirely offline, restricting the model's adaptability to time-varying wind speed data that may exhibit distribution shifts.

To address these limitations, several promising directions will be pursued in future research. First, advanced multi-objective optimization techniques—such as multi-fidelity optimization, surrogate-assisted search, and state-of-the-art evolutionary algorithms—will be incorporated to reduce computational overhead and accelerate architecture search within large-scale hybrid spaces. Second, explainable artificial intelligence (XAI) methods, including gradient-based attribution analysis and attention visualization, will be integrated to enhance model interpretability and transparency, thereby revealing the intrinsic reasoning mechanisms underlying wind speed prediction. Third, an online adaptive neural architecture search framework will be developed to enable dynamic model updating, allowing effective adaptation to evolving wind speed characteristics and potential distribution drifts in real-world scenarios. In addition, the model will be extended to multi-step-ahead forecasting via sequence-to-sequence architectures, and multi-source data fusion will be adopted to further improve robustness, ultimately promoting the practical implementation of the proposed method in industrial wind farm systems.

VI. Conclusion

In this paper, we have proposed a novel multi-objective automated hybrid deep learning framework called MOCB-WSP for wind speed prediction by using CNN-BiLSTM with an efficient fixed-length hybrid encoding mechanism. In the proposed MOCB-WSP, CNN and BiLSTM are combined to extract temporal features of wind speed series, while NSGA-II is introduced to jointly optimize the neural architectures and hyperparameters in a unified search space, overcoming the heavy reliance on expert experience in manual model design. To demonstrate the performance of MOCB-WSP, we compare it with five representative manually designed models and one neuroevolution-based method on three real-world wind farm datasets. The experimental results show the superior performance of MOCB-WSP in terms of prediction accuracy (MAE, RMSE, MAPE, and R) based on (24) and model lightweightness (trainable parameters) defined in (25), achieving an optimal accuracy-complexity trade-off for practical wind speed forecasting.

References

[1] Global Wind Energy Council, "Global wind report 2025," Global Wind Energy Council, Lisbon, Portugal, Tech. Rep., Apr. 2025. [Online]. Available: https://www.gwec.net/reports/globalwindreport/2025

[2] V. Khare, S. Nema, and P. Baredar, "Solar–wind hybrid renewable energy system: A review," *Renewable and Sustainable Energy Reviews*, vol. 58, pp. 23–33, 2016.

[3] K. Yunus, T. Thiringer, and P. Chen, "Arima-based frequency-decomposed modeling of wind speed time series," *IEEE Transactions on Power Systems*, vol. 31, no. 4, pp. 2546–2556, 2015.

[4] Y. Zhang, J. Han, G. Pan, Y. Xu, and F. Wang, "A multi-stage predicting methodology based on data decomposition and error correction for ultra-short-term wind energy prediction," *Journal of Cleaner Production*, vol. 292, p. 125981, 2021.

[5] B. Park and J. Hur, "Accurate short-term power forecasting of wind turbines: The case of jeju island's wind farm," *Energies*, vol. 10, no. 6, p. 812, 2017.

[6] M. G. De Giorgi, S. Campilongo, A. Ficarella, and P. M. Congedo, "Comparison between wind power prediction models based on wavelet decomposition with least-squares support vector machine (ls-svm) and artificial neural network (ann)," *Energies*, vol. 7, no. 8, pp. 5251–5272, 2014.

[7] H. Demolli, A. S. Dokuz, A. Ecemis, and M. Gokcek, "Wind power forecasting based on daily wind speed data using machine learning algorithms," *Energy Conversion and Management*, vol. 198, p. 111823, 2019.

[8] S. Liu, H. Ji, and M. C. Wang, "Nonpooling convolutional neural network forecasting for seasonal time series with trends," *IEEE transactions on neural networks and learning systems*, vol. 31.

[9] H. Liu, X. Mi, and Y. Li, "Smart multi-step deep learning model for wind speed forecasting based on variational mode decomposition, singular spectrum analysis, lstm network and elm," *Energy Conversion and Management*, vol. 159, pp. 54–64, 2018.

[10] X. Luo, J. Sun, L. Wang, W. Wang, W. Zhao, J. Wu, J.-H. Wang, and Z. Zhang, "Short-term wind speed forecasting via stacked extreme learning machine with generalized correntropy," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 11, pp. 4963–4971, 2018.

[11] Y. Huang, Z. Zhang, X. Li, J. Xie, and K. Y. Lee, "Layered-vine copula-based wind speed prediction using spatial correlation and meteorological influence," *IEEE Transactions on Instrumentation and Measurement*, vol. 72, pp. 1–12, 2023.

[12] Z. Zheng and Z. Zhang, "A stochastic recurrent encoder decoder network for multistep probabilistic wind power predictions," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 7, pp. 9565–9578, 2024.

[13] Z. Zhang, L. Lin, S. Gao, J. Wang, and H. Zhao, "Wind speed prediction in china with fully-convolutional deep neural network," *Renewable and Sustainable Energy Reviews*, vol. 201, p. 114623, 2024.

[14] Y. Gao, J. Wang, and H. Yang, "A multi-component hybrid system based on predictability recognition and modified multi-objective optimization for ultra-short-term onshore wind speed forecasting," *Renewable Energy*, vol. 188, pp. 384–401, 2022.

[15] M. Li, K. Zhang, M. Kou, and Y. Ma, "An offshore wind speed forecasting system based on feature enhancement, deep time series clustering, and extended lstm," *Energy*, vol. 333, p. 137335, 2025.

[16] Y. Li, H. Wang, J. Yan, C. Ge, S. Han, and Y. Liu, "Ultra-short-term wind power forecasting based on the strategy of "dynamic matching and online modeling"," *IEEE Transactions on Sustainable Energy*, vol. 16, no. 1, pp. 107–123, 2025.

[17] M. Khodayar and J. Wang, "Spatio-temporal graph deep neural network for short-term wind speed forecasting," *IEEE Transactions on Sustainable Energy*, vol. 10, no. 2, pp. 670–681, 2018.

[18] X. Song, X. Xie, Z. Lv, G. G. Yen, W. Ding, J. Lv, and Y. Sun, "Efficient evaluation methods for neural architecture search: A survey," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 12, pp. 5990–6011, 2024.

[19] B. Bischl, M. Binder, M. Lang, T. Pielok, J. Richter, S. Coors, J. Thomas, T. Ullmann, M. Becker, A.-L. Boulesteix *et al.*, "Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 13, no. 2, p. e1484, 2023.

[20] Y. Sun, G. G. Yen, and Z. Yi, "Evolving unsupervised deep neural networks for learning meaningful representations," *IEEE Transactions on Evolutionary Computation*, vol. 23, no. 1, pp. 89–103, 2018.

[21] A. Baldominos, Y. Saez, and P. Isasi, "Evolutionary convolutional neural networks: An application to handwriting recognition," *Neurocomputing*, vol. 283, pp. 38–52, 2018.

[22] X. Dong, M. Tan, A. W. Yu, D. Peng, B. Gabrys, and Q. V. Le, "Autohas: Differentiable hyper-parameter and architecture search," *arXiv preprint arXiv:2006.03656*, vol. 4, no. 5, 2020.

[23] L. Zimmer, M. Lindauer, and F. Hutter, "Auto-pytorch: Multi-fidelity metalearning for efficient and robust autodl," *IEEE transactions on pattern analysis and machine intelligence*, vol. 43, no. 9, pp. 3079–3090, 2021.

[24] Z. Lu, I. Whalen, Y. Dhebar, K. Deb, E. D. Goodman, W. Banzhaf, and V. N. Boddeti, "Multiobjective evolutionary design of deep convolutional neural networks for image classification," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 2, pp. 277–291, 2021.

[25] Y. Xue, P. Jiang, C. Zhu, M. Zhou, M. Wahib, and M. Gabbouj, "A pairwise comparison relation-assisted multiobjective evolutionary neural architecture search method with multipopulation mechanism," *IEEE*

Transactions on Systems, Man, and Cybernetics: Systems, vol. 56, no. 2, pp. 1274–1287, 2026.

[26] Y. Xue, P. Jiang, C. Zhu, Y. Zhang, R. Cheng, K. Gao, and D. Gong, "A multiobjective evolutionary algorithm based on bipopulation with uniform sampling for neural architecture search," *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–15, 2026.

[27] Y. Yang, Q. Zhu, J. Luo, K.-C. Wong, Q. Lin, and J. Li, "Trnas: A training-free robust neural architecture search," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 2336–2345.

[28] H.-N. Wei, G.-Q. Zeng, K.-D. Lu, G.-G. Geng, and J. Weng, "Moar-cnn: Multi-objective adversarially robust convolutional neural network for sar image classification," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 9, no. 1, pp. 57–74, 2024.

[29] S. Han, W. Song, J. Yan, N. Zhang, H. Wang, C. Ge, and Y. Liu, "Integrating intra-seasonal oscillations with numerical weather prediction for 15-day wind power forecasting," *IEEE Transactions on Power Systems*, 2025.

[30] X. Liang, Q. Gu, and X. You, "Wpformer: A spatial-temporal graph transformer with auto-correlation for wind power forecasting," *IEEE Transactions on Sustainable Energy*, vol. 16, no. 3, pp. 1491–1503, 2024.

[31] J. Zhang, Y. Chen, H. Pan, L. Cao, and C. Li, "Constrained deep reinforcement transfer learning for short-term forecasting of wind discrepancies at ocean stations," *Neurocomputing*, vol. 624, p. 129491, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231225001638

[32] S. M. J. Jalali, G. J. Osório, S. Ahmadian, M. Lotfi, V. M. Campos, M. Shafie-khah, A. Khosravi, and J. P. Catalão, "New hybrid deep neural architectural search-based ensemble reinforcement learning strategy for wind power forecasting," *IEEE Transactions on Industry Applications*, vol. 58, no. 1, pp. 15–27, 2021.

[33] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 2002.

[34] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[35] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.

[36] S. Izquierdo, J. Guerrero-Viu, S. Hauns, G. Miotto, S. Schrodi, A. Biedenkapp, T. Elsken, D. Deng, M. Lindauer, and F. Hutter, "Bag of baselines for multi-objective joint neural architecture search and hyperparameter optimization," in *8th ICML Workshop on Automated Machine Learning (AutoML)*, 2021.

[37] M.-E. Nilsback and A. Zisserman, "Automated flower classification over a large number of classes," in *2008 Sixth Indian conference on computer vision, graphics & image processing*. IEEE, 2008, pp. 722–729.

[38] S. S. Mostafa, F. Mendonca, A. G. Ravelo-Garcia, G. G. Juliá-Serdá, and F. Morgado-Dias, "Multi-objective hyperparameter optimization of convolutional neural network for obstructive sleep apnea detection," *IEEE Access*, vol. 8, pp. 129 586–129 599, 2020.

[39] J. Liu, R. Cheng, and Y. Jin, "Bi-fidelity evolutionary multiobjective search for adversarially robust deep neural architectures," *Neurocomputing*, vol. 550, p. 126465, 2023.

[40] Z. Zhao, S. Yun, L. Jia, J. Guo, Y. Meng, N. He, X. Li, J. Shi, and L. Yang, "Hybrid vmd-cnn-gru-based model for short-term forecasting of wind power considering spatio-temporal features," *Engineering Applications of Artificial Intelligence*, vol. 121, p. 105982, 2023.

[41] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.

[42] Y. Guo, Y. Li, X. Qiao, Z. Zhang, W. Zhou, Y. Mei, J. Lin, Y. Zhou, and Y. Nakanishi, "Bilstm multitask learning-based combined load forecasting considering the loads coupling relationship for multienergy system," *IEEE Transactions on Smart Grid*, vol. 13, no. 5, pp. 3481–3492, 2022.

[43] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE transactions on evolutionary computation*, vol. 6, no. 2, pp. 182–197, 2002.

[44] G.-A. Vargas-Hakim, E. Mezura-Montes, and H.-G. Acosta-Mesa, "A review on convolutional neural network encodings for neuroevolution," *IEEE Transactions on Evolutionary Computation*, vol. 26, no. 1, pp. 12–27, 2021.

[45] J.-C. Huang, G.-Q. Zeng, G.-G. Geng, J. Weng, K.-D. Lu, and Y. Zhang, "Differential evolution-based convolutional neural networks: An automatic architecture design method for intrusion detection in industrial control systems," *Computers & Security*, vol. 132, p. 103310, 2023.

[46] B. Wang, B. Xue, and M. Zhang, "Surrogate-assisted particle swarm optimization for evolving variable-length transferable blocks for image classification," *IEEE transactions on neural networks and learning systems*, vol. 33, no. 8, pp. 3727–3740, 2021.

[47] C. Huang, H. R. Karimi, P. Mei, D. Yang, and Q. Shi, "Evolving long short-term memory neural network for wind speed forecasting," *Information Sciences*, vol. 632, pp. 390–410, 2023.

[48] X. Qin, L. Yuan, X. Dong, S. Zhang, and H. Shi, "Short term wind speed prediction based on ceesmdan and improved seagull optimization kernel extreme learning machine," *Earth Science Informatics*, vol. 18, no. 1, p. 141, 2025.

[49] X. Hu, L. Chu, J. Pei, W. Liu, and J. Bian, "Model complexity of deep learning: A survey," *Knowledge and Information Systems*, vol. 63, no. 10, pp. 2585–2619, 2021.

[50] S. Harbola and V. Coors, "One dimensional convolutional neural network architectures for wind prediction," *Energy Conversion and Management*, vol. 195, pp. 70–75, 2019.

[51] Y. Zhou and X. Fan, "Mrgs-lstm: a novel multi-site wind speed prediction approach with spatio-temporal correlation," *Frontiers in Energy Research*, vol. 12, p. 1427587, 2024.

[52] H. Zhang, J. Wang, Y. Qian, and Q. Li, "Point and interval wind speed forecasting of multivariate time series based on dual-layer lstm," *Energy*, vol. 294, p. 130875, 2024.

[53] A. Lawal, S. Rehman, L. M. Alhems, and M. M. Alam, "Wind speed prediction using hybrid 1d cnn and blstm network," *IEEe Access*, vol. 9, pp. 156 672–156 679, 2021.

[54] P. Mode, C. Demartino, C. T. Georgakis, and N. D. Lagaros, "Short-term extreme wind speed forecasting using dual-output lstm-based regression and classification model," *Journal of Wind Engineering and Industrial Aerodynamics*, vol. 259, p. 106035, 2025.

[55] F. Shahid, A. Zameer, and M. Muneeb, "A novel genetic lstm model for wind power forecast," *Energy*, vol. 223, p. 120069, 2021.